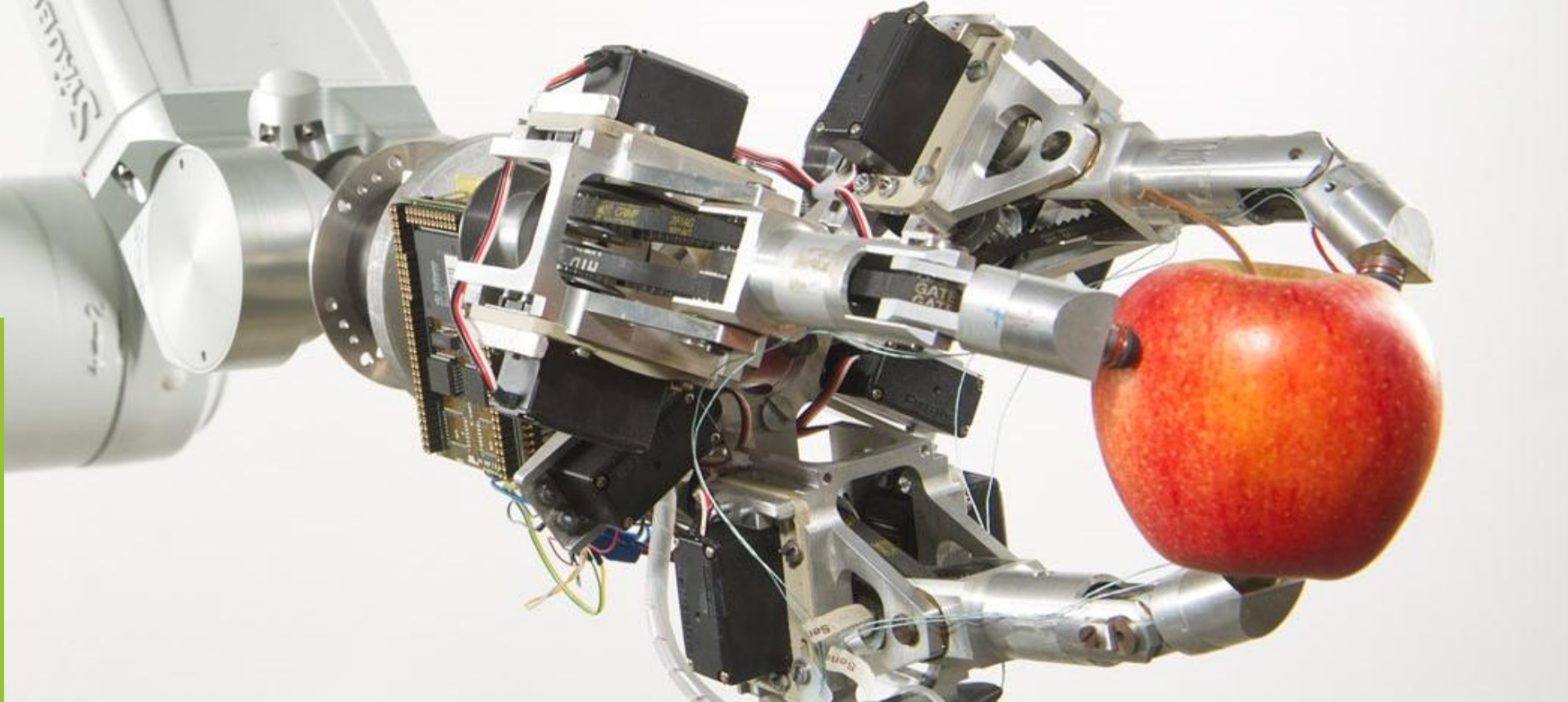




Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

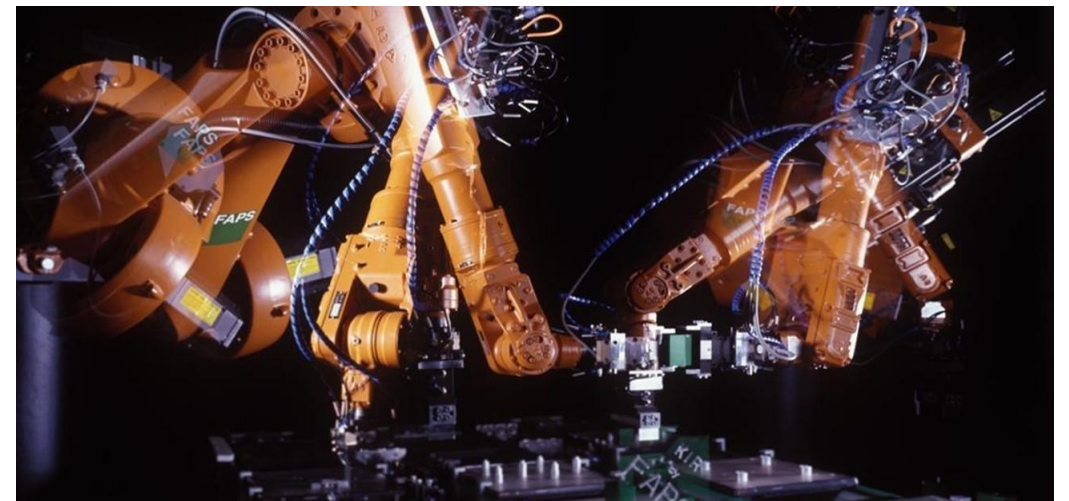
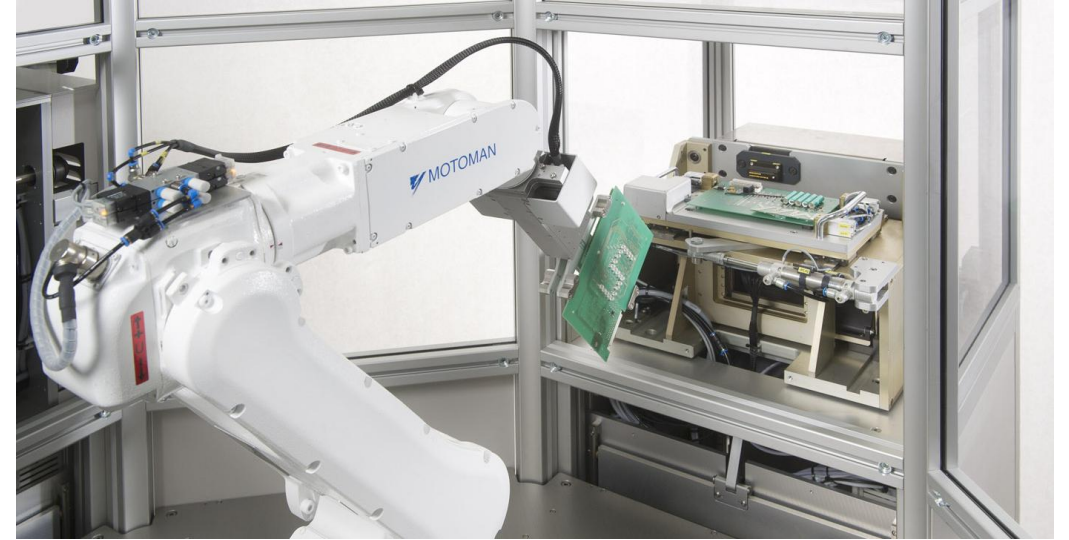
Grundlagen der Robotik (GdR)

VE04: Roboterkinematik und Dynamik
Vorlesung im SS 2026

Im Fach Grundlagen der Robotik werden die fachspezifischen Grundlagen zur Konzeption, Implementierung und Realisierung von Robotersystemen vermittelt.

VE	Thema
----	-------

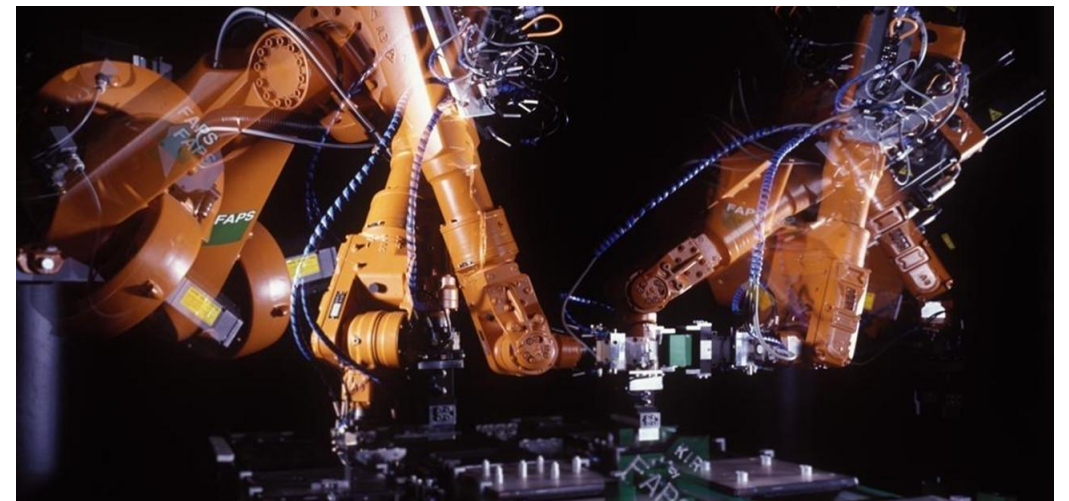
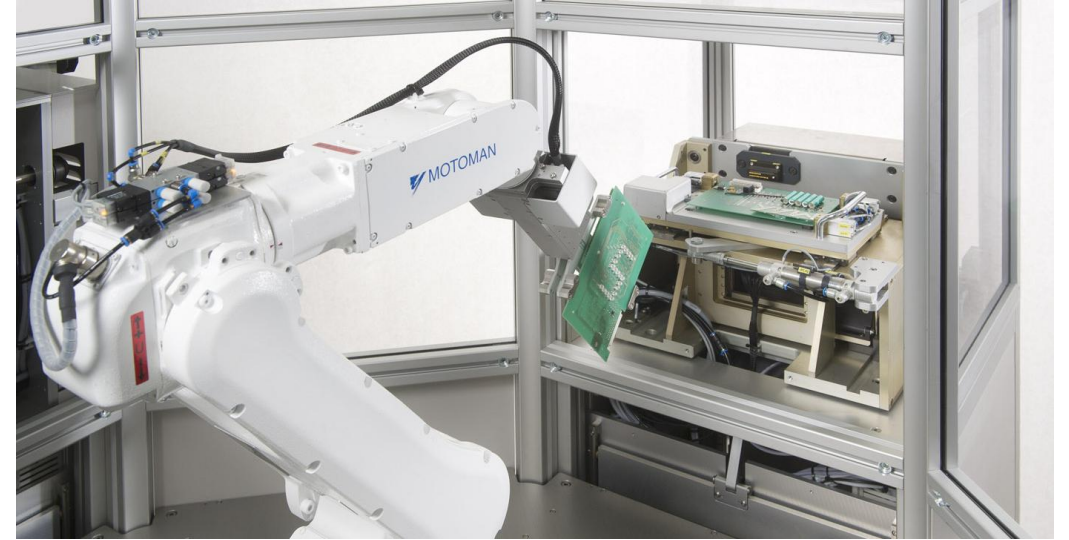
- | | |
|----|--|
| 1 | --- entfällt --- |
| 2 | Einführung und Grundlagen der Industrierobotik & Neuartige Konzepte und Kinematiken in der Robotik |
| 3 | Steuerung - Sensorik – Aktorik |
| 4 | Roboterkinematik |
| 5 | Roboterdynamik |
| 6 | Bahnplanung und Regelung |
| 7 | Grundlagen des Robot Operating System (ROS) |
| 8 | ROS - Erweiterte Konzepte und Tools |
| 9 | Simulation und Planung |
| 10 | Rechnersehen |
| 11 | Maschinelles Lernen |
| 12 | Seilrobotik |
| 14 | Service-, Pflege- und Medizinrobotik |



Die zugehörige Übung setzt sich aus Tafel- und Hausübungen zusammen und fokussiert unter anderem die praktische Arbeit mit dem Robot Operating System.

UE	Thema
----	-------

- | | |
|----|--|
| 1 | --- entfällt --- |
| 2 | --- entfällt --- |
| 3 | Hallenführung in Kleingruppen |
| 4 | Steuerung - Sensorik – Aktorik |
| 5 | Roboterkinematik |
| 6 | Roboterdynamik |
| 7 | Bahnplanung und Regelung (verschoben auf 14.07.) |
| 8 | Hallenführung in Kleingruppen |
| 9 | Grundlagen des Robot Operating System (ROS) |
| 10 | ROS - Erweiterte Konzepte und Tools |
| 11 | Simulation und Planung |
| 12 | Rechnersehen |
| 13 | Maschinelles Lernen |



Kompakte Darstellungsformen bieten den Vorteil der einfacheren, computergestützten Berechnung und können das Auftreten von Singularitäten vermeiden.

■ **Orientierungsdarstellung durch Drehvektoren und -winkel**

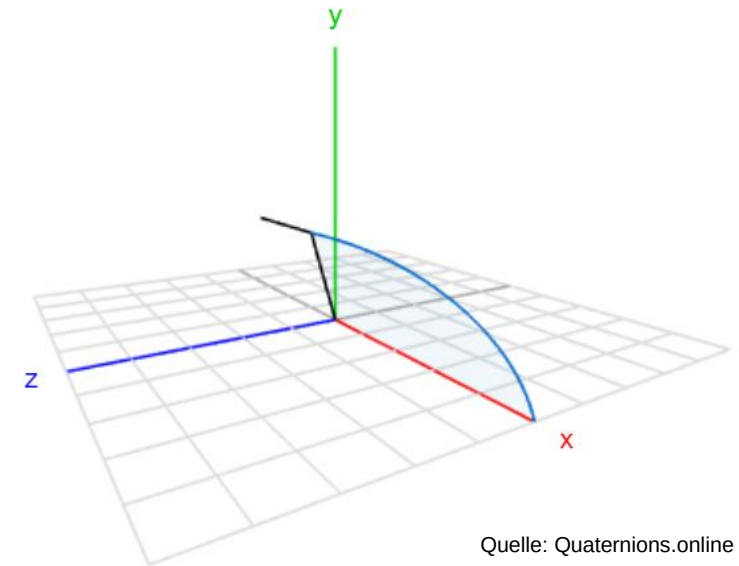
- Angle-Axis-Darstellung
- Quaternionen

■ **Vorwärts- und Inverse Kinematik**

- Denavit-Hartenberg-Parameter (DH-Parameter)
- Vorwärtskinematik
- Jacobi-Matrix
- Inverse Kinematik

■ **Dynamik von Robotersystemen**

- Lagrange-Verfahren
- Newton-Euler-Schema



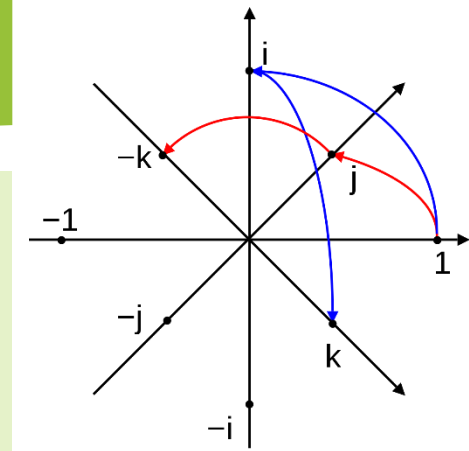
Quelle: Quaternions.online

Die Vorlesungseinheit „Roboterkinematik und Dynamik“ soll an einfachen Beispielen die Berechnung der Geschwindigkeiten sowie der wirkenden Kräfte und Momente vermitteln.

Die **kinematischen und dynamischen Beschreibungen** von Robotersystemen bilden die **Grundlagen** für **alle Aufgaben** in der Robotik.

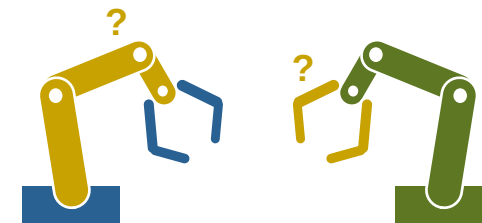
Nach dem Besuch dieser Vorlesungseinheit sollten Sie in der Lage sein:

- Die **Orientierungsdarstellung über Quaternionen** darzustellen,
- entlang einer kinematischen Kette die **DH-Parameter** des Systems zu notieren,
- die **Systematik der Vorwärts- und inversen Kinematik** zu kennen,
- die **Bedeutung und Funktion der Jacobi-Matrix** erläutern zu können,
- **alternative Berechnungsformen der inversen Kinematik** zu kennen sowie
- **grundlegende Prinzipien der Roboterdynamik** gegenüberzustellen.



$$\begin{aligned} ij &= k \\ ji &= -k \\ ij &= -ji \end{aligned}$$

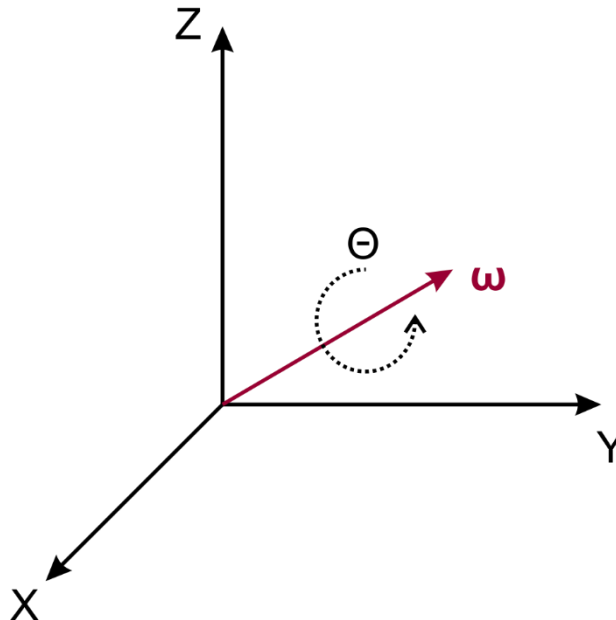
Quelle: Wikipedia



Die Angle-Axis (bzw. Achse-Winkel)-Darstellung ermöglicht die Angabe einer Orientierung mithilfe des Winkels θ und des Einheitsvektors ω .

Orientierungsdarstellung durch Achse-Winkel

- Einzelner Winkel θ in Kombination mit einem Einheitsvektor ω durch den Ursprung
- Rotation erfolgt um den Winkel θ um eine durch den Vektor $\omega = (\omega_x \ \omega_y \ \omega_z)^T$ definierte Achse
- Schreibweise: $\theta\omega$, $(\theta\omega_x \ \theta\omega_y \ \theta\omega_z)^T$ oder (ω, θ)
- Das Paar der Größen (ω, θ) wird auch als Drehzeiger der Drehung bezeichnet
- Die Drehzeiger (ω, θ) und $(-\omega, -\theta)$ beschreiben dieselbe Drehung



Einheitsquaternionen lösen das Problem von auftretenden Singularitäten auf Grund von Freiheitsgradverlusten bei der Orientierungsdarstellung mit Euler-Winkeln.

- Sehr nützlich in der Robotik, da keine Singularitäten; jedoch **nur Darstellung von Rotationen** (und nicht Translationen) möglich

Darstellung über komplexe Zahlen

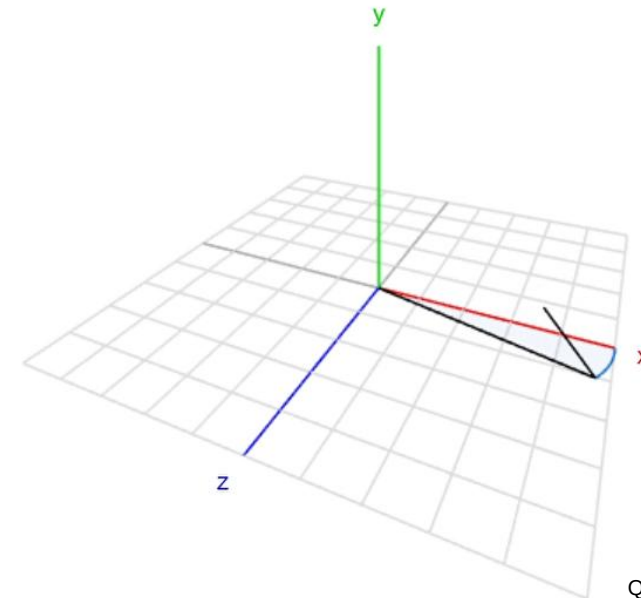
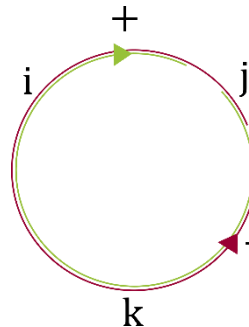
$$\mathbf{q} = q_0 + q_1i + q_2j + q_3k$$

- Erweiterter reeller Zahlenbereich
- Viergliedrige, komplexe Zahl: Realteil q_0 mit reeller Einheit gleich eins, drei Imaginärteile q_1 , q_2 und q_3 mit imaginären Einheiten i , j , k

Regeln für die Produkte der imaginären Einheiten

$$ii = jj = kk = -1 = ijk$$

$$\begin{array}{lll} ij = k, & jk = i, & ki = j \\ ji = -k, & kj = -i, & ik = -j \end{array}$$



Quelle: Quaternions.online

Die Koeffizienten der Real- und Imaginärteile lassen sich auch in der Vektorschreibweise darstellen.

- Sehr nützlich in der Robotik, da keine Singularitäten; jedoch **nur Darstellung von Rotationen** (und nicht Translationen) möglich

Verknüpfung mit Vektorrechnung

$$\mathbf{q} = (s, \mathbf{v})$$

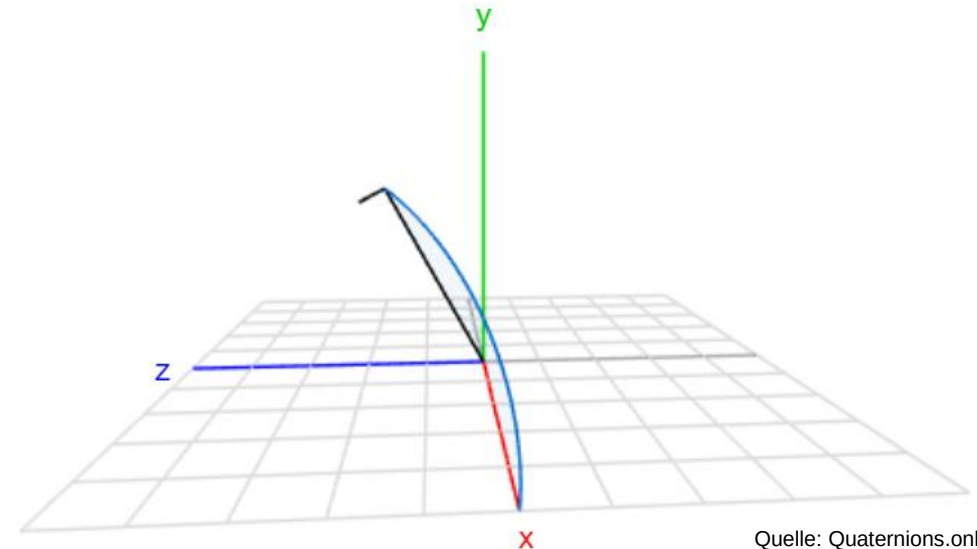
- Realteil q_0 wird als Skalarteil s (Rotationswinkel) und die drei Imaginärteile q_1, q_2 und q_3 als Vektorteil $\mathbf{v}$ (Rotationsachse) der Quaternion $\mathbf{q}$ definiert.
- Die Koeffizienten q_0, q_1, q_2 und q_3 sind Skalare; sie entsprechen bei Einheitsquaternionen den sog. **Euler-Parametern**.

$$\mathbf{q} = \begin{bmatrix} q_0 \\ - \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} q_0 \\ \vec{\mathbf{q}} \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ \omega_x \sin(\theta/2) \\ \omega_y \sin(\theta/2) \\ \omega_z \sin(\theta/2) \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ \boldsymbol{\omega} \sin(\theta/2) \end{bmatrix}$$

- Die Quaternion $\mathbf{q}$ ist hierbei eine Einheitsquaternion mit dem Betrag gleich eins:

$$|\mathbf{q}| = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2} = 1$$

- Die Einheitsquaternion $\mathbf{q}$ einer Quaternion $\mathbf{a}$ ist $\mathbf{q} = \frac{\mathbf{a}}{|\mathbf{a}|}$



Quelle: Quaternions.online

Beim Rechnen mit Quaternionen müssen definierte Regeln beachtet werden.

Addition zweier Quaternionen (kommutativ)

- Nullelement für die Addition: $0 = 0 + 0i + 0j + 0k$
- Addition durch separates Addieren der entsprechenden Komponenten:

$$\mathbf{c} = \mathbf{a} + \mathbf{b} = \overbrace{(a_0 + b_0)}^{c_0} + \overbrace{(a_1 + b_1)}^{c_1} i + \overbrace{(a_2 + b_2)}^{c_2} j + \overbrace{(a_3 + b_3)}^{c_3} k$$

Multiplikation zweier Quaternionen

- Nullelement für die Multiplikation: $I = 1 + 0i + 0j + 0k$
- Multiplikation durch Verwendung von Skalar- und Vektorprodukt (Multiplikationsoperator $\circ$):

$$\mathbf{c} = \mathbf{a} \circ \mathbf{b} = \overbrace{a_0 b_0 - a_1 b_1 - a_2 b_2 - a_3 b_3}^{c_0} + \overbrace{(a_0 b_1 + a_1 b_0 + a_2 b_3 - a_3 b_2)}^{c_1} i + \\ + \underbrace{(a_0 b_2 + a_2 b_0 + a_3 b_1 - a_1 b_3)}_{c_2} j + \underbrace{(a_0 b_3 + a_3 b_0 + a_1 b_2 - a_2 b_1)}_{c_3} k$$

bzw.

$$\begin{bmatrix} a_0 \\ \vec{a} \end{bmatrix} \circ \begin{bmatrix} b_0 \\ \vec{b} \end{bmatrix} = \begin{bmatrix} a_0 b_0 - \vec{a} \vec{b} \\ a_0 \vec{b} + b_0 \vec{a} + \vec{a} \times \vec{b} \end{bmatrix}$$

- Assoziativ $\mathbf{a} \circ (\mathbf{b} \circ \mathbf{c}) = (\mathbf{a} \circ \mathbf{b}) \circ \mathbf{c} = \mathbf{a} \circ \mathbf{b} \circ \mathbf{c}$
- Distributiv $\mathbf{a} \circ (\mathbf{b} + \mathbf{c}) = \mathbf{a} \circ \mathbf{b} + \mathbf{a} \circ \mathbf{c}$
- Nicht kommutativ $\mathbf{a} \circ \mathbf{b} \neq \mathbf{b} \circ \mathbf{a}$

Quaternionen bieten eine zeit- und speicherplatzreduzierte Möglichkeit zur Transformation zwischen mehreren Koordinatensystemen.

Rotationsdarstellung mittels einer Einheitsquaternion q

- Definition eines Vektors p als Quaternion mit dem Skalarteil $q_0 = 0$:

$$\begin{aligned} p &= (p_x \ p_y \ p_z)^T \\ p &= p_x i + p_y j + p_z k \end{aligned}$$

- Operation $q \circ p \circ \bar{q}$ beschreibt eine Rotation des Vektors p um die durch $(q_1 \ q_2 \ q_3)^T$ definierte Achse, z. B. eine Drehung vom K_1 ins K_0 :

$$p^{(0)} = q \circ p^{(1)} \circ \bar{q}$$

- Konjugierte Quaternion $\bar{q}$: Die drei Imaginärteile haben das umgekehrte Vorzeichen

$$\begin{aligned} \bar{q} &= q_0 - q_1 i - q_2 j - q_3 k \\ q \circ \bar{q} &= q_0^2 + q_1^2 + q_2^2 + q_3^2 \end{aligned}$$

→ Für Einheitsquaternionen: $q^{-1} = \bar{q}$

- Multiplikation einzelner Rotationen durch Multiplikation der jeweiligen Quaternionen

$$\begin{aligned} c = a \circ b &= \underbrace{a_0 b_0 - a_1 b_1 - a_2 b_2 - a_3 b_3}_{c_0} + \underbrace{(a_0 b_1 + a_1 b_0 + a_2 b_3 - a_3 b_2)}_{c_1} i + \underbrace{(a_0 b_2 + a_2 b_0 + a_3 b_1 - a_1 b_3)}_{c_2} j + \underbrace{(a_0 b_3 + a_3 b_0 + a_1 b_2 - a_2 b_1)}_{c_3} k \end{aligned}$$

Einheitsquaternionen können in verschiedene Darstellungen umgerechnet werden, dazu zählen die Rotationsdarstellung mit Rotationsmatrizen und die Achse-Winkel-Darstellung.

Umwandlung in Angle-Axis

- Umrechnung einer Einheitsquaternion q in die Angle-Axis Darstellung

$$q = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} q_0 \\ \vec{q} \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ \omega_x \sin(\theta/2) \\ \omega_y \sin(\theta/2) \\ \omega_z \sin(\theta/2) \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ \omega \sin(\theta/2) \end{bmatrix}$$

Gegebene Einheitsquaternion: $q = 0,9659 + 0,2588i + 0,2588j = \begin{bmatrix} 0,9659 \\ 0,2588 \\ 0,2588 \\ 0 \end{bmatrix}$

Rotationswinkel $\theta = 30^\circ$

Vektor $\omega = (\omega_x \ \omega_y \ \omega_z)^\top = (1, 1, 0)^\top$

Umwandlung in Rotationsmatrizen

- Umrechnung einer Einheitsquaternion q (mit $q = q_0 + q_1i + q_2j + q_3k$) in die Darstellung einer Rotationsmatrix

$$R = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix}$$

- Bestimmung einer Einheitsquaternion q (d. h. ihrer Koeffizienten q_0, q_1, q_2 und q_3) auf Basis einer Rotationsmatrix

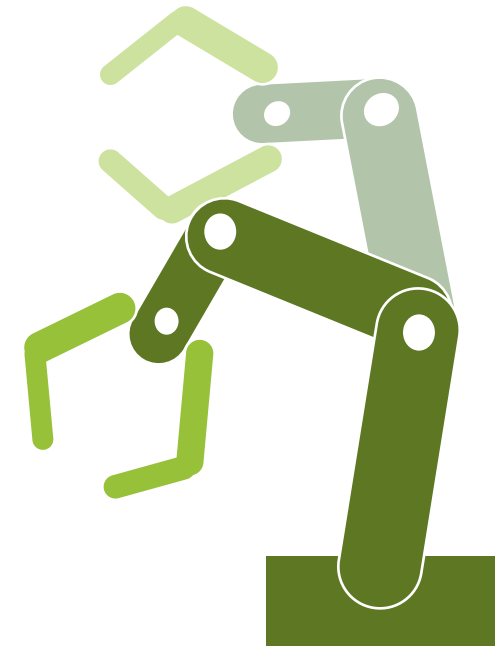
mit $R = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix}$

$$q_0 = \frac{1}{2} \sqrt{1 + R_{11} + R_{22} + R_{33}}, \quad q_1 = \frac{R_{32} - R_{23}}{4q_0},$$

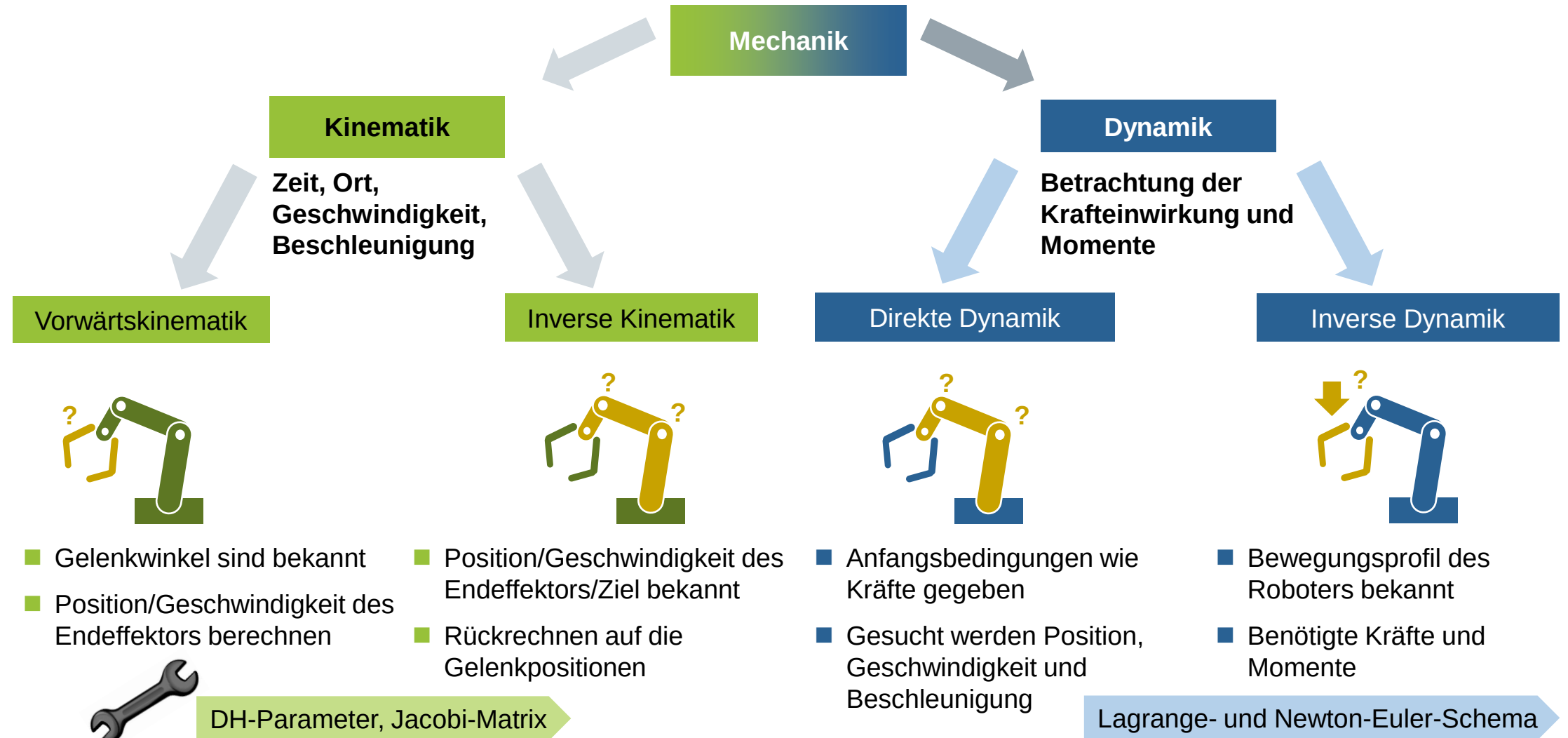
$$q_2 = \frac{R_{13} - R_{31}}{4q_0}, \quad q_3 = \frac{R_{21} - R_{12}}{4q_0}$$

Die direkte und inverse Kinematik dienen primär der Bestimmung von Endeffektor- bzw. Gelenkwinkel-Positionen und -Geschwindigkeiten.

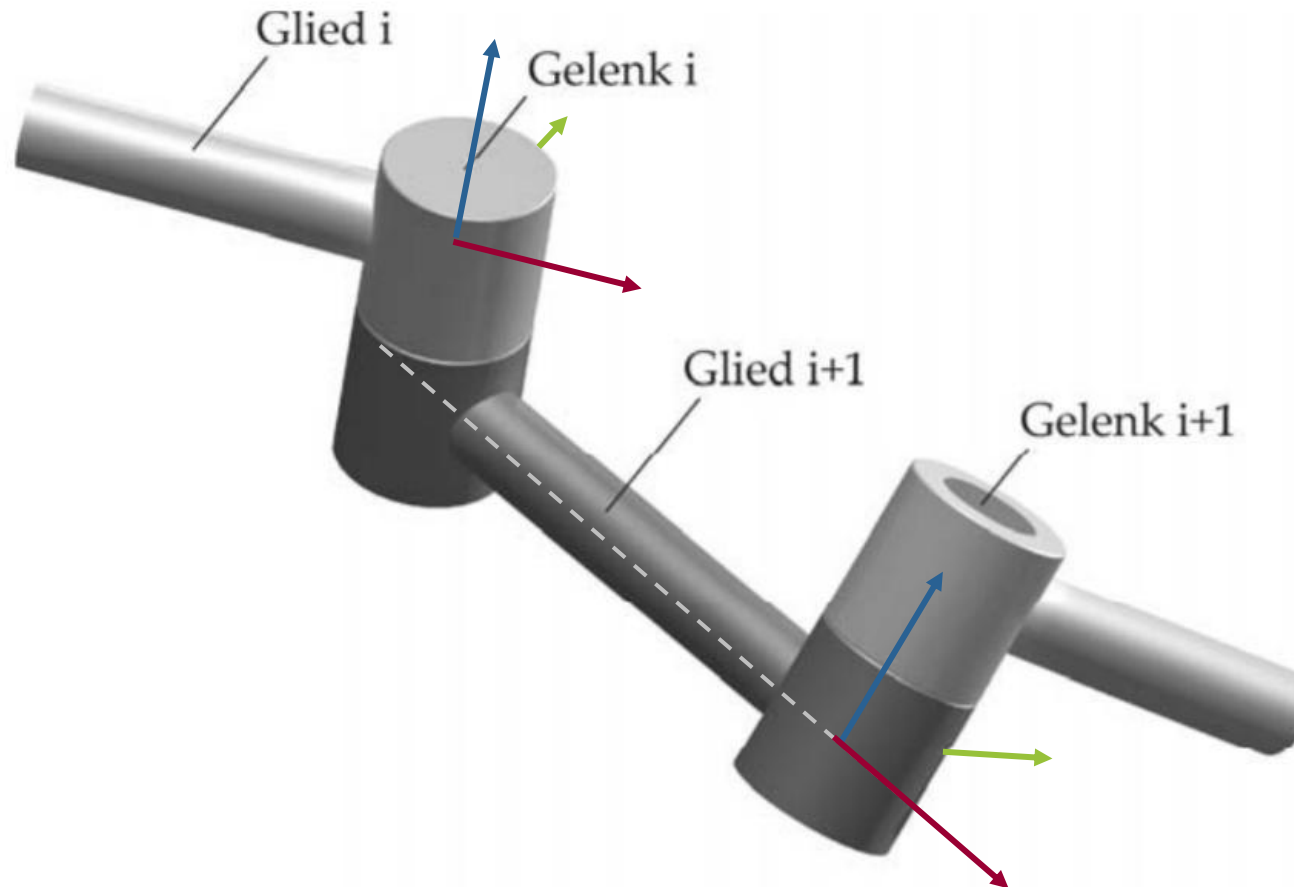
- Orientierungsdarstellung durch Drehvektoren und -winkel
 - Angle-Axis-Darstellung
 - Quaternionen
- **Vorwärts- und Inverse Kinematik**
 - Denavit-Hartenberg-Parameter (DH-Parameter)
 - Vorwärtskinematik
 - Jacobi-Matrix
 - Inverse Kinematik
- Dynamik von Robotersystemen
 - Lagrange-Verfahren
 - Newton-Euler-Schema



Die Mechanik teilt sich in die geometrische Beschreibung der Bewegung, die Kinematik, und in die erweiterte Betrachtung durch die Krafteinwirkung, die Dynamik, auf.



Für die Beschreibung von Transformationen durch Denavit-Hartenberg-Parameter müssen definierte Regeln befolgt werden.



Vorgaben für die Verwendung von Denavit-Hartenberg-(DH)-Parametern:

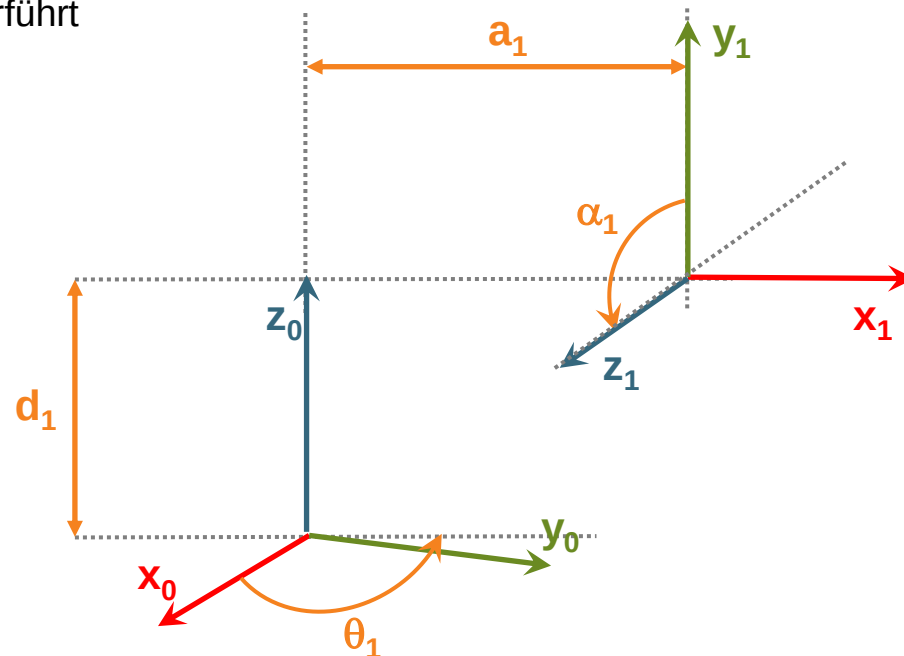
- Vorhergehendes Gelenk i muss gegeben sein
- Ursprung des KS_{i+1} liegt im Schnittpunkt der Gelenkachse $i+1$ mit der gemeinsamen Normalen von i und $i+1$
- Z-Achse wird so gewählt, dass sie der Bewegungsachse des Gelenks entspricht
- X-Achse entlang der gemeinsamen Normalen von i und $i+1$ und zeigt im Regelfall von i weg
- Y-Achse wird so ergänzt, dass es sich um ein rechtshändiges Koordinatensystem handelt

Quelle: Brutscheck und Bückner

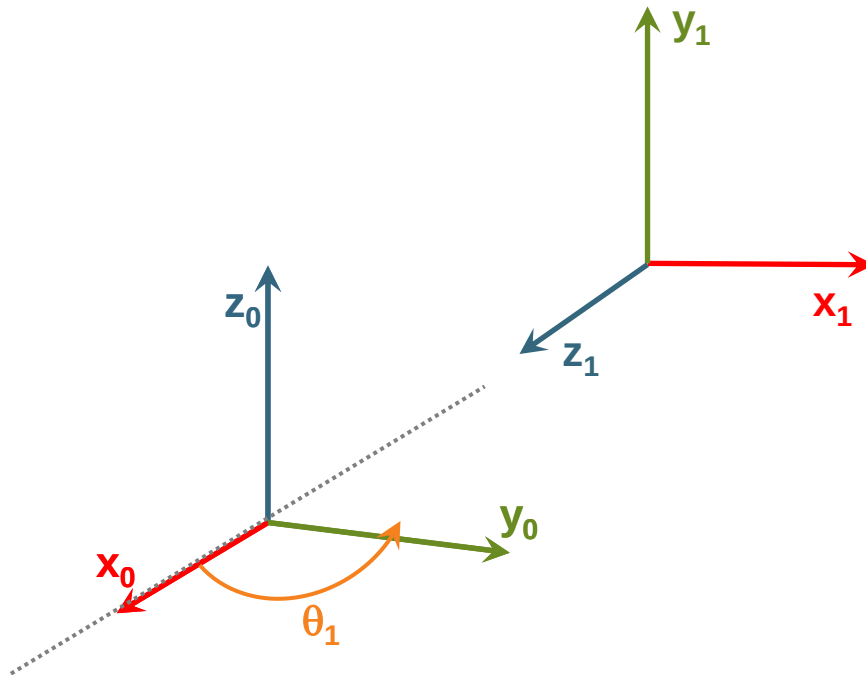
Mithilfe der vier Denavit-Hartenberg-Parameter lassen sich zwei benachbarte Koordinatensysteme K_i in K_{i-1} ineinander überführen.

Vier Denavit-Hartenberg-Parameter (DH-Parameter):

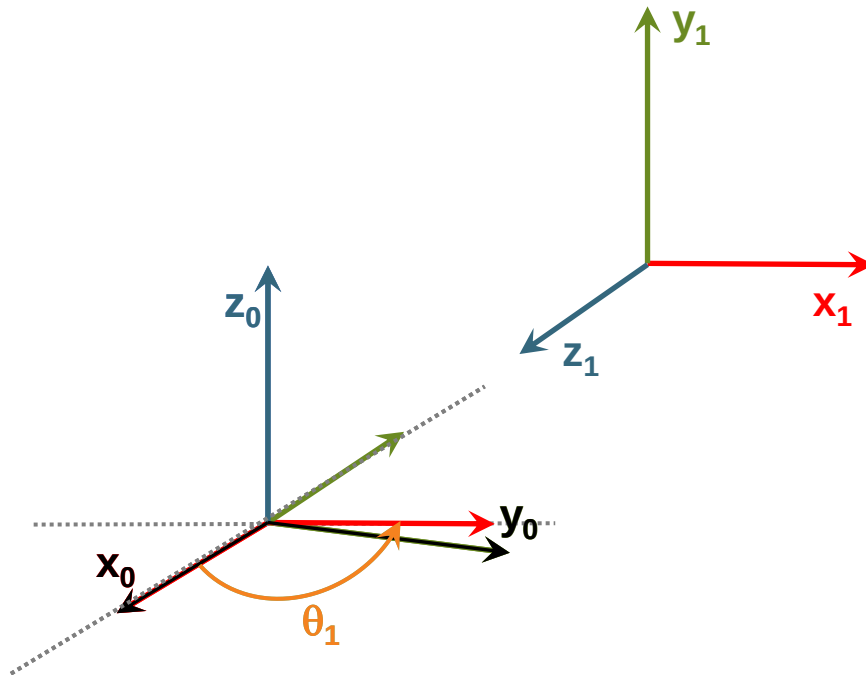
- θ_i : absoluter Drehwinkel um Gelenk i , d.h. um z_{i-1} , der x_{i-1} in x_i dreht
- d_i : Translation vom Ursprung von K_{i-1} entlang z_{i-1} , die den Abstand der Ursprünge von K_{i-1} und K_i minimal macht (gemessen wird in Richtung z_{i-1})
- a_i : Translation in Richtung x_i , so dass der Ursprung von K_{i-1} möglichst nahe an den Ursprung von K_i kommt bzw. mit dem Ursprung von K_i zusammenfällt. (a_i ist immer positiv)
- α_i : Drehwinkel um x_i , der z_{i-1} in z_i überführt



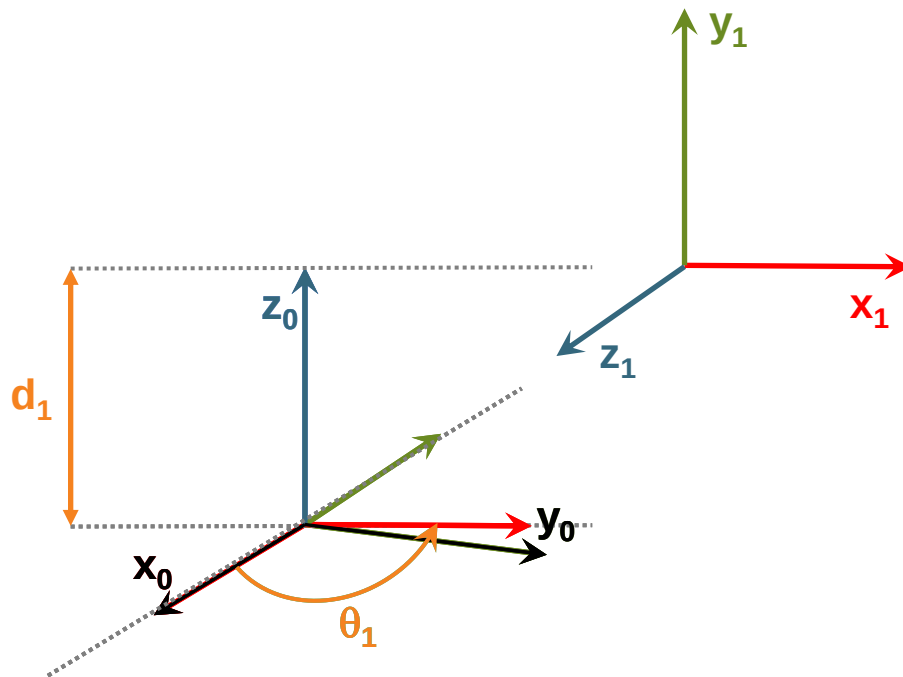
Durch eine Animation wird die Überführung zweier benachbarter Koordinatensysteme mithilfe der DH-Parameter veranschaulicht.



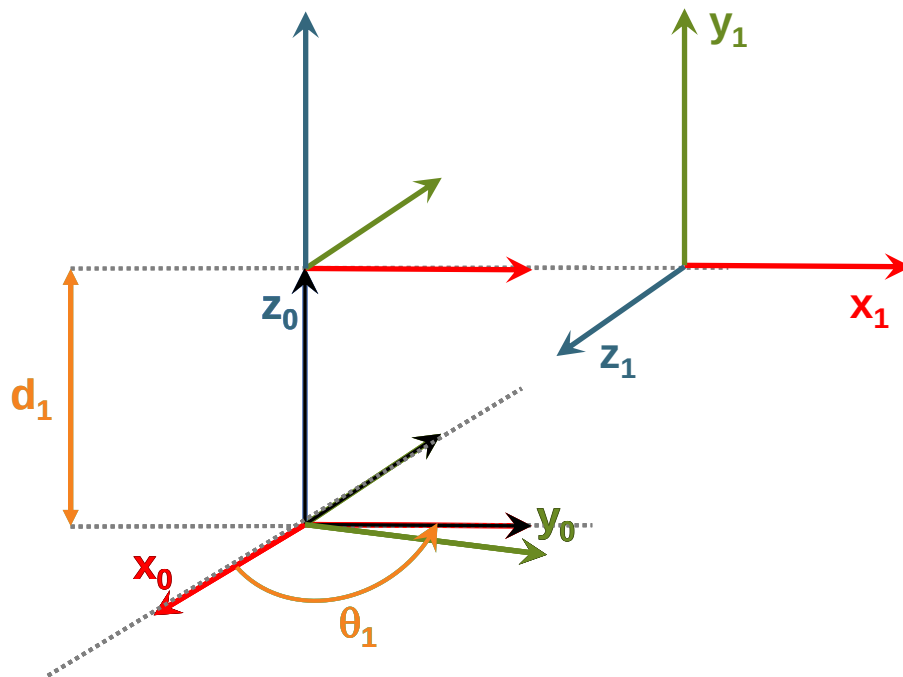
Durch eine Animation wird die Überführung zweier benachbarter Koordinatensysteme mithilfe der DH-Parameter veranschaulicht.



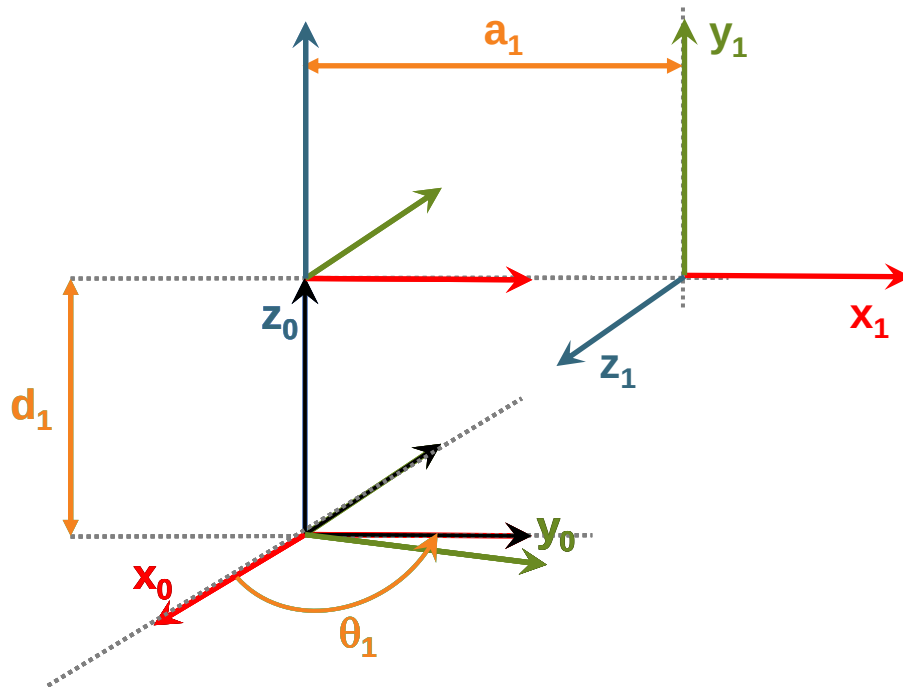
Durch eine Animation wird die Überführung zweier benachbarter Koordinatensysteme mithilfe der DH-Parameter veranschaulicht.



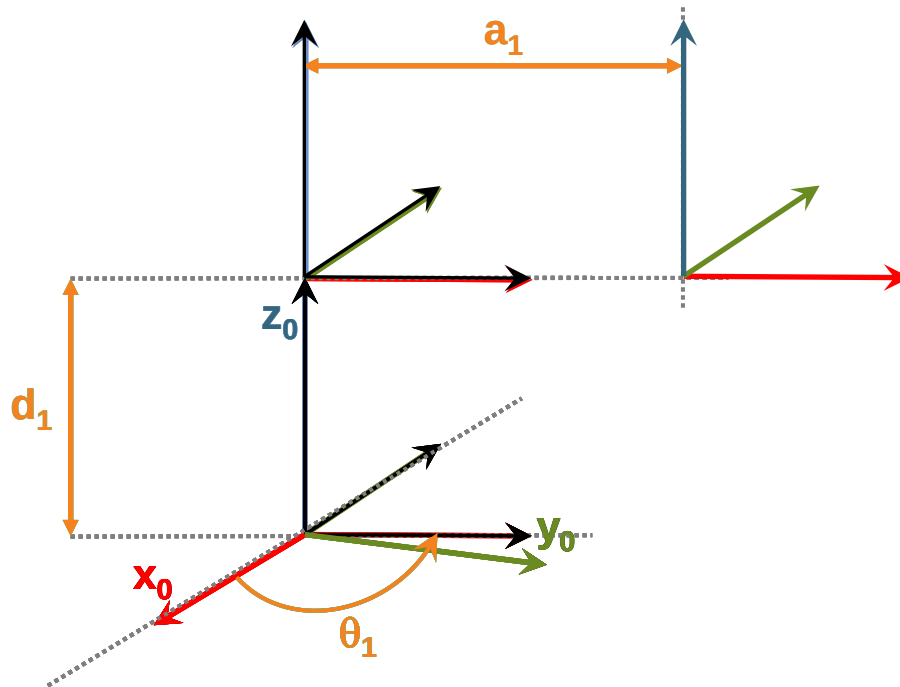
Durch eine Animation wird die Überführung zweier benachbarter Koordinatensysteme mithilfe der DH-Parameter veranschaulicht.



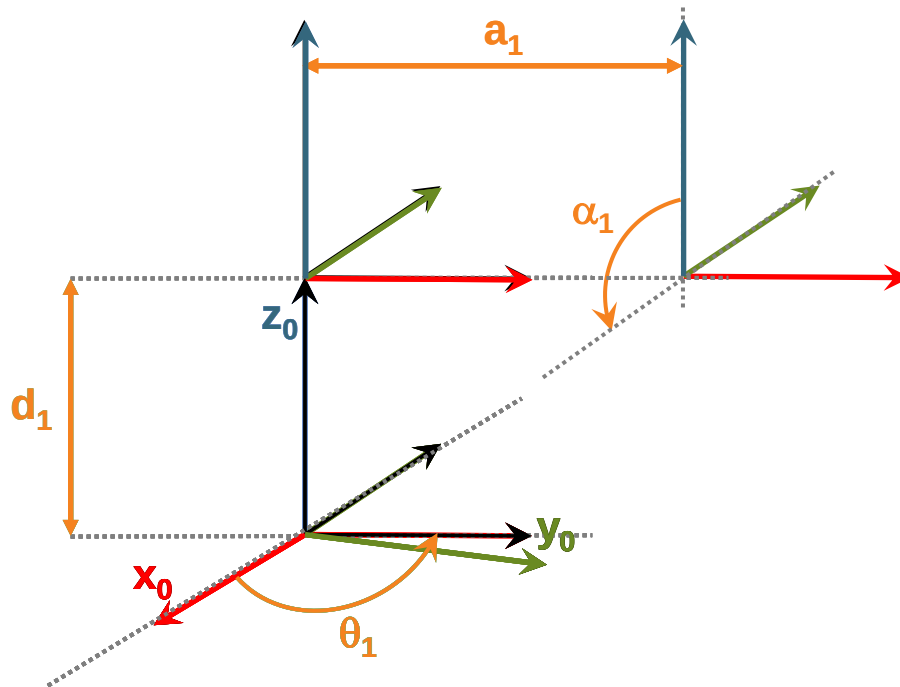
Durch eine Animation wird die Überführung zweier benachbarter Koordinatensysteme mithilfe der DH-Parameter veranschaulicht.



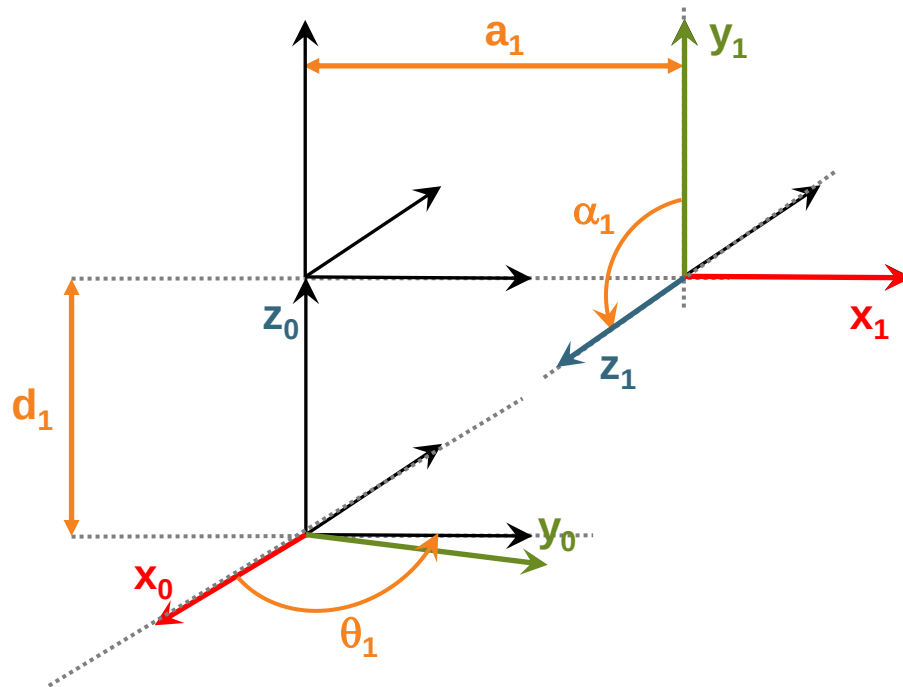
Durch eine Animation wird die Überführung zweier benachbarter Koordinatensysteme mithilfe der DH-Parameter veranschaulicht.



Durch eine Animation wird die Überführung zweier benachbarter Koordinatensysteme mithilfe der DH-Parameter veranschaulicht.



Durch eine Animation wird die Überführung zweier benachbarter Koordinatensysteme mithilfe der DH-Parameter veranschaulicht.



Gelenktyp	θ_i	d_i	a_i	α_i
Rot.	θ_i	konst.	konst.	konst.
Trans.	konst.	d_i	konst.	konst.

Mit Hilfe der DH-Parameter kann eine Vorwärtstransformation durch Kettenmultiplikation von einzelnen Transformationsmatrizen aufgestellt werden.

$${}^t_0\mathbf{T}(q) = {}^1_0\mathbf{T}(q_1) \cdot {}^2_1\mathbf{T}(q_2) \cdot {}^t_2\mathbf{T}(q_t) =$$

$$= \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 & a_1 \cos \theta_1 \\ \sin \theta_1 & \cos \theta_1 & 0 & a_1 \sin \theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & a_2 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & a_2 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta_3 & -\sin \theta_3 & 0 & a_3 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 & 0 & a_3 \sin \theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix}$$

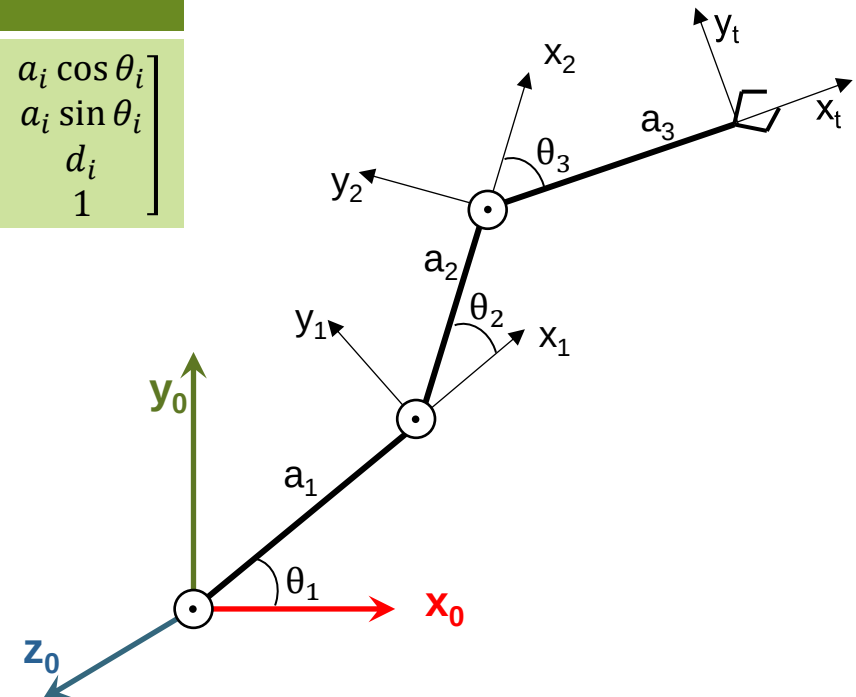
Geltende Konvention für die Matrixschreibweise der DH-Parameter

$${}_{i-1}^i\mathbf{T} = \begin{bmatrix} x_i^{(i-1)} & y_i^{(i-1)} & z_i^{(i-1)} & p_{i-1,i}^{(i-1)} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

	θ_i	d_i	a_i	α_i
Achse 1	$\theta_1 + \pi/2$	0	a_1	0
Achse 2	$\theta_2 + 0$	0	a_2	0
Achse 3	$\theta_3 - \pi/2$	0	a_3	0

Offsets

Abweichung zwischen reeller Achsstellung und Position in Software.



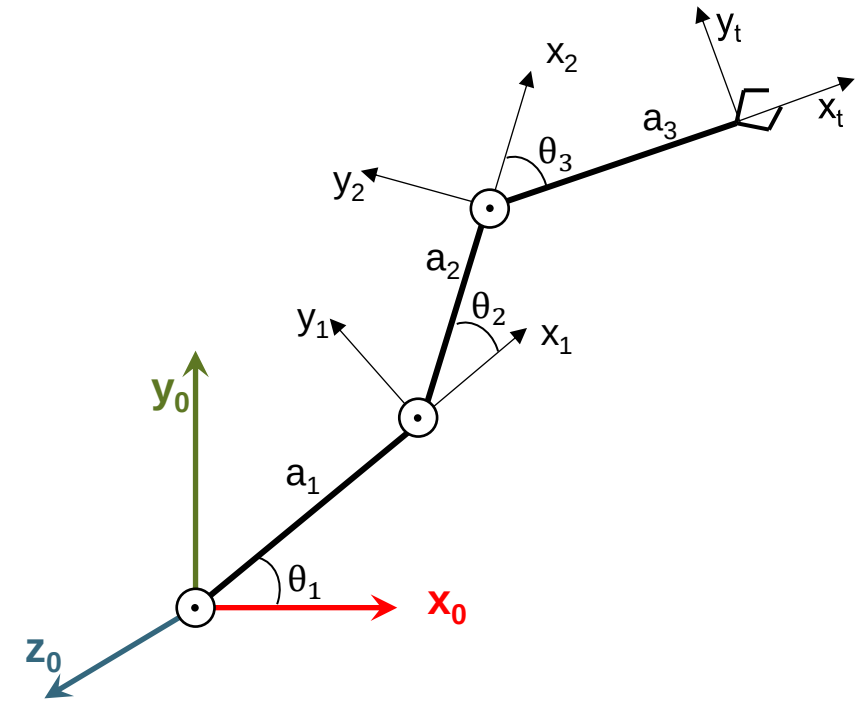
Die direkte Kinematik bzw. Vorwärtstransformation wird zur Bestimmung der relativen Orientierung und Position des Endeffektors zum Basiskoordinatensystem verwendet.

Vorwärtstransformation bzw. direkte Kinematik

- Gelenkwinkel gegeben
- Eindeutige Abbildung von Gelenkkoordinaten auf Weltkoordinaten (kartesische Koordinaten)
- Berechnung über Kettenmultiplikation mehrerer Transformationen

$${}^t_0\mathbf{T}(q) = {}^1_0\mathbf{T}(q_1) \cdot {}^2_1\mathbf{T}(q_2) \cdot {}^t_2\mathbf{T}(q_t)$$

- Einfach lösbar für offene kinematische Ketten
- Komplex für geschlossene kinematische Ketten



Konvention der DH-Parameter

- θ_1 : absoluter Drehwinkel um z_0 , der x_0 in x_1 dreht
- d_1 : Translation vom Ursprung von K_0 entlang z_0
- a_1 : Translation in Richtung x_1
- α_1 : Drehwinkel um x_1 , der z_0 in z_1 überführt

	θ_i	d_i	a_i	α_i
Achse 1	$\theta_1 + \pi/2$	0	a_1	0
Achse 2	$\theta_2 + 0$	0	a_2	0
Achse 3	$\theta_3 - \pi/2$	0	a_3	0

Die Jacobi-Matrix setzt Endeffektor-Geschwindigkeiten in Relation zu Gelenkwinkel-Geschwindigkeiten und ist abhängig von der Gelenkwinkelkonfiguration.

- Für die Verwendung der **Jacobi-Matrix** in der **Vorwärtskinematik** wird die Repräsentation über eine **Vektordarstellung** benötigt (bspw. Roll-Pitch-Yaw)
- Für die **Endeffektorpose** $x(t)$ zum Zeitpunkt t gilt:

$$x(t) = f(\theta(t))$$
- Daraus folgt für die **Geschwindigkeit des Endeffektors** $\dot{x}(t)$ bzw. für die **Geschwindigkeit der Gelenkwinkel** $\dot{\theta}(t)$:

$$\dot{x}(t) = \frac{\partial f(\theta(t))}{\partial t} = J_A(\theta(t)) \cdot \dot{\theta}(t)$$

math. Definition der Jacobi-Matrix

$$J_A(\theta) = \frac{\partial f(\theta)}{\partial \theta} = \text{grad } f = \nabla f(\theta) = \begin{bmatrix} \frac{\partial f_1}{\partial \theta_1}(\theta) & \dots & \frac{\partial f_1}{\partial \theta_n}(\theta) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial \theta_1}(\theta) & \dots & \frac{\partial f_m}{\partial \theta_n}(\theta) \end{bmatrix}$$

Es gibt mehrere Formen zur Berechnung der Jacobi-Matrix:

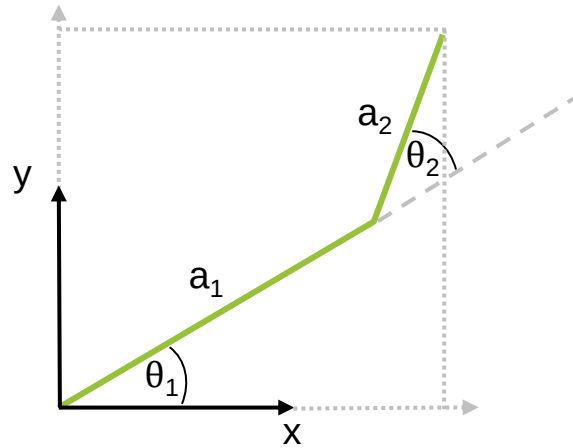
- Geometrische Berechnung der Jacobi-Matrix möglich, jedoch Wissen über den Aufbau notwendig
- Numerische Berechnung ist allgemeingültig

Singularitäten treten auf, wenn die Jacobi-Matrix singulär wird!

Ausblick auf die Dynamik

Die **Jacobi-Matrix** stellt außerdem den Zusammenhang zwischen den **Drehmomenten** der Gelenke und den **Kräften** und **Momente** am Endeffektor her.

Die Kinematik beschreibt auch in der Robotik die geometrische Bewegung von Körpern in Abhängigkeit von Zeit, Ort, Geschwindigkeit und Beschleunigung.



Vorwärtskinematik

- Gesucht: Geschwindigkeit des Endeffektors
- Resultierend aus: $x(t) = f(\theta(t)) \rightarrow \begin{pmatrix} x \\ y \end{pmatrix} = f \begin{pmatrix} \theta_1 \\ \theta_2 \end{pmatrix}$ mit x = Endeffektorposition
 θ = Gelenkwinkel
- Ergibt sich: $\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = J_f(\theta) \cdot \begin{pmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{pmatrix}$ mit $\dot{x}$ = Endeffektorgeschwindigkeit
 $\dot{\theta}$ = Gelenkwinkelgeschwindigkeit

$${}^2_0T = \begin{bmatrix} c(\theta_1 + \theta_2) & -s(\theta_1 + \theta_2) & 0 & a_1 c\theta_1 + a_2 c(\theta_1 + \theta_2) \\ s(\theta_1 + \theta_2) & c(\theta_1 + \theta_2) & 0 & a_1 s\theta_1 + a_2 s(\theta_1 + \theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$x = a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2)$$

$$y = a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2)$$

Ableitung

$$\dot{x} = -a_1 \dot{\theta}_1 \sin \theta_1 - a_2 (\dot{\theta}_1 + \dot{\theta}_2) \sin(\theta_1 + \theta_2)$$

$$\dot{y} = a_1 \dot{\theta}_1 \cos \theta_1 + a_2 (\dot{\theta}_1 + \dot{\theta}_2) \cos(\theta_1 + \theta_2)$$

Jacobi-Matrix

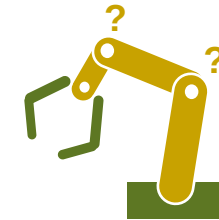
$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \underbrace{\begin{pmatrix} -a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2) \\ a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) \end{pmatrix}}_{J_1(\theta)} \underbrace{\begin{pmatrix} -a_2 \sin(\theta_1 + \theta_2) \\ a_2 \cos(\theta_1 + \theta_2) \end{pmatrix}}_{J_2(\theta)} \cdot \begin{pmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{pmatrix}$$

Werte der Gelenkwinkelpositionen und -geschwindigkeiten einsetzen!

Bei der umgekehrten Problemstellung, der Rückwärtstransformation bzw. inversen Kinematik, ist die Bestimmung der Gelenkkoordinaten für eine Endeffektor-Pose nicht immer eindeutig.

Rückwärtstransformation bzw. inverse Kinematik

- Bestimmung der Werte der Gelenkkoordinaten θ
- Gegebene Position und Orientierung des Endeffektors (relativ zur Basis)
- Erfordert nichtlineare Gleichungen
- Komplex für offene kinematische Ketten
- Einfach lösbar für geschlossene kinematische Ketten



Möglichkeit, dass keine (Endeffektor-Pose außerhalb des Arbeitsbereichs des Roboters), mehrere oder unendlich viele Lösungen existieren!

Geschlossene Lösungen (Architektur abhängig)

- schneller als numerische Verfahren, einfachere Bestimmung aller Lösungen, roboterabhängig, existieren nicht für alle kinematischen Roboterstrukturen

Algebraische Methode

$${}^w_0T = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} = P^{(w)}$$

Aufstellen eines Gleichungssystems aus den Matrixeinträgen

Geometrische Methode

Zerlegung des Problems:

- inverse Positionierung
- inverse Orientierung

Numerische Lösungsverfahren (Architektur unabhängig)

$$\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = J^{-1}(\theta) \cdot \begin{bmatrix} \dot{x}_x \\ \dot{x}_y \end{bmatrix}$$

- Iterationsverfahren (Iterative Methods)
- Eliminationsverfahren (Symbolic Elimination Methods)
- Fortsetzungsverfahren (Continuation Methods)

Das Beispiel des einfachen Roboters mit zwei Rotationsgelenken in der Ebene zeigt die Bestimmung der singulären Roboter-Posen anhand seiner Jakobi-Matrix.

$$J(\theta) = \begin{bmatrix} -a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2) & -a_2 \sin(\theta_1 + \theta_2) \\ a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) & a_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

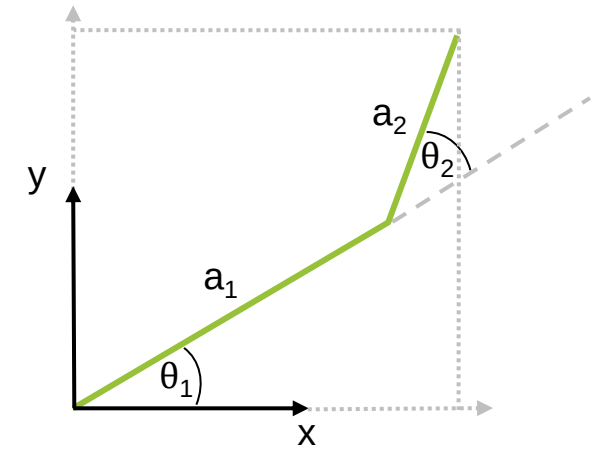
Nutzung der Additionstheoreme und $\sin^2(\theta) + \cos^2(\theta) = 1$

$$\det(J(\theta)) = a_1 a_2 [-\sin \theta_1 \cos(\theta_1 + \theta_2) + \cos \theta_1 \sin(\theta_1 + \theta_2)] = a_1 a_2 \sin \theta_2$$

$\det J(\theta) = 0 \rightarrow$ Singularität,

falls $\theta_2 = 0$ (ausgestreckter Roboterarm, d. h. äußerer Rand des Arbeitsraums)

oder $\theta_2 = \pi$ (eingeklappter Roboterarm, d. h. innerer Rand des Arbeitsraums)



Inverse Problemstellung

$$\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = J^{-1}(\theta) \cdot \begin{bmatrix} \dot{p}_x \\ \dot{p}_y \end{bmatrix} \quad \text{mit} \quad J^{-1} = \frac{1}{a_1 a_2 \sin \theta_2} \begin{bmatrix} a_2 \cos(\theta_1 + \theta_2) & a_2 \sin(\theta_1 + \theta_2) \\ -a_1 \cos \theta_1 - a_2 \cos(\theta_1 + \theta_2) & -a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2) \end{bmatrix}$$

$$\text{und} \quad \dot{x} = \begin{bmatrix} \dot{p}_x \\ \dot{p}_y \end{bmatrix} = \begin{bmatrix} 1 \text{ m/s} \\ 0 \end{bmatrix}$$

$$\dot{\theta}_1 = \frac{\cos(\theta_1 + \theta_2)}{a_1 \sin \theta_2} \quad \dot{\theta}_2 = \frac{-a_1 \cos \theta_1 - a_2 \cos(\theta_1 + \theta_2)}{a_1 a_2 \sin \theta_2}$$

Falls θ_2 gegen Null geht, gehen die Achsgeschwindigkeiten gegen unendlich.

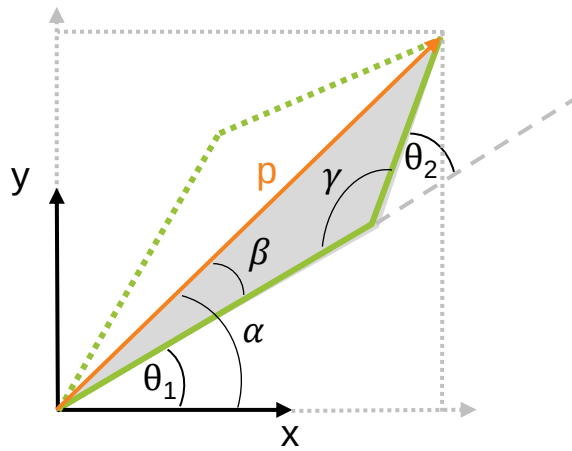
Inverse einer 2x2 Matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Unter bestimmten Voraussetzungen an die Roboterarchitektur existieren geschlossene Lösungen, die beispielsweise geometrisch gelöst werden können.

Geometrische Lösung der inversen Kinematik



$$\begin{aligned} \theta_1 &= \alpha - \beta \\ \theta_2 &= \pi - \gamma \end{aligned}$$

- Zur Berechnung wird der Kosinussatz herangezogen
- Einführen der Hilfswinkel α, β, γ

$$\theta_2 = \pm \arccos \left(\frac{x^2 + y^2 - a_1^2 - a_2^2}{2 a_1 a_2} \right)$$

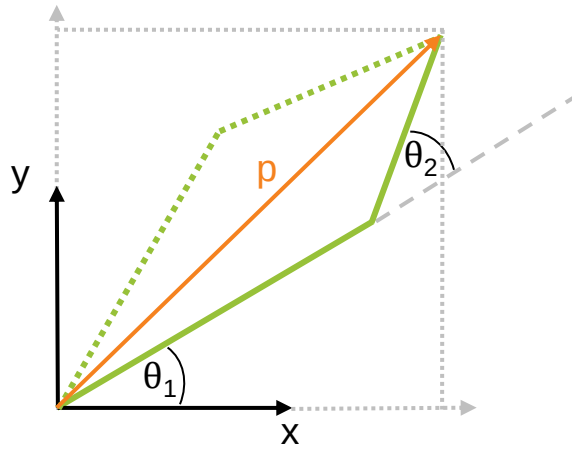
$$\alpha = \arctan2(x, y)$$

$$\beta = \pm \arccos \left(\frac{(x^2 + y^2) - a_1^2 - a_2^2}{2 a_1 \sqrt{x^2 + y^2}} \right)$$

- Ohne Einschränkung ergeben sich zwei Lösungen für arccos

Unter bestimmten Voraussetzungen an die Roboterarchitektur existieren geschlossene Lösungen, die teilweise algebraisch gelöst werden können.

Algebraische Lösung der inversen Kinematik



- Koeffizientenvergleich zwischen den Matrixeinträgen der Endeffektorlage und der Gesamttransformationsmatrix

Endeffektorlage $t = [x, y, \varphi]$

$${}^t_0T(t) = \begin{bmatrix} c(\varphi) & -s(\varphi) & 0 & x \\ s(\varphi) & c(\varphi) & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^t_0T(\theta) = \begin{bmatrix} c(\theta_1 + \theta_2) & -s(\theta_1 + \theta_2) & 0 & a_1c\theta_1 + a_2c(\theta_1 + \theta_2) \\ s(\theta_1 + \theta_2) & c(\theta_1 + \theta_2) & 0 & a_1s\theta_1 + a_2s(\theta_1 + \theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

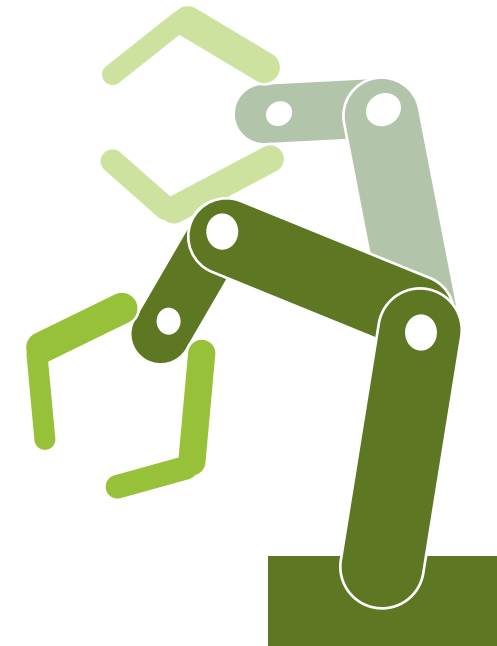
Beispiel:

$$\rightarrow x = a_1c\theta_1 + a_2c(\theta_1 + \theta_2)$$

$$\rightarrow y = a_1s\theta_1 + a_2s(\theta_1 + \theta_2)$$

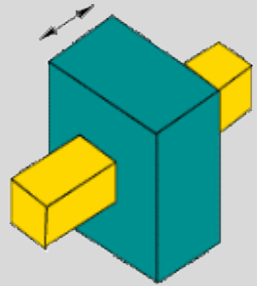
Die dynamische Betrachtung erweitert die Kinematik um Aspekte der angreifenden Kräfte und Momenten, die auf das Robotersystem wirken.

- Orientierungsdarstellung durch Drehvektoren und -winkel
 - Angle-Axis-Darstellung
 - Quaternionen
- Vorwärts- und Inverse Kinematik
 - Denavit-Hartenberg-Parameter (DH-Parameter)
 - Vorwärtskinematik
 - Jacobi-Matrix
 - Inverse Kinematik
- **Dynamik von Robotersystemen**
 - Lagrange-Verfahren
 - Newton-Euler-Schema



Der statische Zusammenhang zwischen den Antriebskräften und den Kräften am Endeffektor kann über die Jacobi-Matrix hergestellt werden.

Antriebskraft Schubgelenk



Prismatic
1 Degree of Freedom

$$\tau_i = J_E^{trans T} \begin{bmatrix} M_E \\ F_E \end{bmatrix}$$

mit τ_i Antriebskraft

M_e, F_e Kraft/Moment welche durch den Endeffektor ausgeübt werden

J_E^{trans} translatorische Jacobi-Matrix

Antriebsmoment Rotationsgelenk



Revolute
1 Degree of Freedom

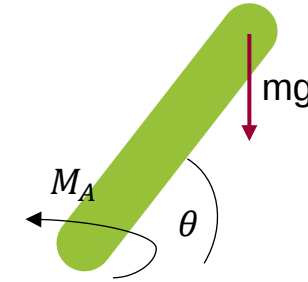
$$\tau_i = J_E^{rot T} \begin{bmatrix} M_E \\ F_E \end{bmatrix}$$

mit τ_i Antriebsmoment

M_e, F_e Kraft/Moment welche durch den Endeffektor ausgeübt werden

J_E^{rot} rotatorische Jacobi-Matrix

Quelle: LaValle



Direkte Dynamik

Geg.: Antriebsmoment τ ,
Anfangsbedingung
 $\theta(t_0), \dot{\theta}(t_0)$

Ges.: Bewegung des
Roboters $\theta, \dot{\theta}, \ddot{\theta}$

Numerische
Integration

Differenzieren

Inverse Dynamik

Geg.: Bewegung des
Roboters $\theta, \dot{\theta}, \ddot{\theta}$

Ges.: erforderliches
Antriebsmoment τ

Algebraische
Auswertung

$$\tau = \theta_A \ddot{\theta} + m g l \sin\theta + \partial\dot{\theta}$$

$$\tau = M(q)\ddot{q} + c(q)\dot{q} + g(q)q + e(\ddot{q}, \dot{q}, q)$$

Das Lagrange-Verfahren basiert auf dem Energieerhaltungssatz, wofür Kräfte und Momente eines Roboters in generalisierten Koordinaten dargestellt werden.

- Masseteilchen werden auf Roboter angenommen, welche sich nicht unabhängig voneinander bewegen können
- Die Lagrange-Methode basiert auf dem Energieerhaltungssatz
- Betrachtung des Gesamtsystems

$$L(q, \dot{q}) = E_{kin}(q, \dot{q}) - E_{pot}(q) \quad \text{mit } q, \dot{q}, \ddot{q}: \text{Vektor der generalisierten Koordinaten (minimale Zahl unabhängiger Koordinaten)}$$

Dynamisch

Kinetische Energie des Systems

Zweites Newton'sches Gesetz

$$F_i = m_i \cdot \ddot{x}_i$$

Nicht-lineare Effekte

z. B. Reibung (wird häufig vernachlässigt)

Gravitation

Potentielle Energie des Systems

$$F_i = m_i \cdot g \cdot y_i$$

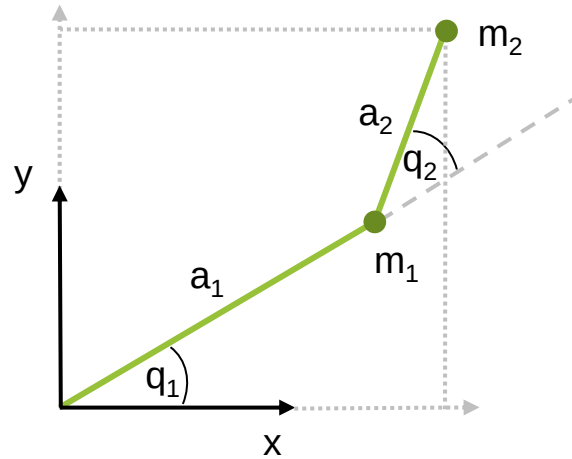
Externe Kräfte

z. B. durch extra Masse am Endeffektor durch Werkstück

$$0 = \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \left(\frac{\partial L}{\partial q_i} \right) \rightarrow \tau = M(q)\ddot{q} + c(q)\dot{q} + g(q)q + e(\ddot{q}, \dot{q}, q)$$

mit τ = Antriebskraft/-moment, $M(q)$ = Massenträgheitsmatrix, $c(q)$ = Zentripetal-/Corioliskräfte, $g(q)$ = Gravitationskräfte, $e(q)$ = nicht-lineare Effekte

Am Beispiel eines vereinfachten, planaren Roboters mit zwei Rotationsgelenken wird die Vorgehensweise des Lagrange-Verfahrens gezeigt.



Es wirkt die Gewichtskraft g , die Reibung wird vernachlässigt

Berechnung der **potentiellen Energie** E_{pot} :

$$E_{\text{pot},1} = mgy_1 = m_1ga_1 \sin q_1$$

$$E_{\text{pot},2} = mgy_2 = m_2g [a_1 \sin q_1 + a_2 \sin(q_1 + q_2)]$$

Berechnung der **kinetischen Energie** E_{kin} :

$$E_{\text{kin},1} = \frac{1}{2}mv^2 = \frac{1}{2}m_1\dot{q}_1^2a_1^2$$

$$E_{\text{kin},2} = \frac{1}{2}mv^2 = \frac{1}{2}m_2a_1^2\dot{q}_1^2 + \frac{1}{2}m_2a_2^2(\dot{q}_1 + \dot{q}_2)^2 + m_2a_1a_2(\dot{q}_1^2 + \dot{q}_1\dot{q}_2)\cos(q_2)$$

$$v^2 = \dot{x}^2 + \dot{y}^2$$

Einsetzen in die **Lagrange-Funktion** L :

$$L(q, \dot{q}) = E_{\text{kin}}(q, \dot{q}) - E_{\text{pot}}(q) = E_{\text{kin},1} + E_{\text{kin},2} - E_{\text{pot},1} - E_{\text{pot},2} =$$

$$= \frac{1}{2}m_1\dot{q}_1^2a_1^2 + \frac{1}{2}m_2a_1^2\dot{q}_1^2 + \frac{1}{2}m_2a_2^2(\dot{q}_1 + \dot{q}_2)^2 + m_2a_1a_2(\dot{q}_1^2 + \dot{q}_1\dot{q}_2)\cos(q_2) - m_1ga_1 \sin q_1 - m_2g [a_1 \sin q_1 + a_2 \sin(q_1 + q_2)]$$

Ableitungen
bilden!

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_1} \right), \left(\frac{\partial L}{\partial q_1} \right), \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_2} \right), \left(\frac{\partial L}{\partial q_2} \right)$$

Durch das Einsetzen der Ableitungen und das Sortieren der Terme entsteht die Bewegungsgleichung für das gesamte System.

- Das **Einsetzen** in die Lagrange-Funktion wird **separat für jedes Gelenk** durchgeführt und jeweils **in einer Zeile** notiert
- Dieses Verfahren funktioniert immer unter der Betrachtung der **Energie des Gesamtsystems**
- Einfaches Aufstellen der Gleichungen
- **Geschlossenes Modell**, das analytisch berechenbar ist
- **Nur Antriebsmomente** werden berücksichtigt

$$\begin{aligned}
 \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} &= \underbrace{\begin{bmatrix} (m_1 + m_2)a_1^2 + m_2a_2^2 + 2m_2a_1a_2\cos(q_2) & m_2a_2^2 + m_2a_1a_2\cos(q_2) \\ m_2a_2^2 + m_2a_1a_2\cos(q_2) & m_2a_2^2 \end{bmatrix}}_{\text{Massenträgheitsmatrix}} \begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix} + \underbrace{\begin{bmatrix} -m_2a_1a_2(2\dot{q}_1\dot{q}_2 + q_2^2\sin(q_2)) \\ m_2a_1a_2(2\dot{q}_1^2\sin(q_2)) \end{bmatrix}}_{\text{Zentripetalkräfte}} + \\
 &+ \underbrace{\begin{bmatrix} (m_1 + m_2)ga_1\cos(q_1) + m_2ga_2\cos(q_1 + q_2) \\ m_2ga_2\cos(q_1 + q_2) \end{bmatrix}}_{\text{Gravitationskomponente}}
 \end{aligned}$$

Der Algorithmus von Newton-Euler ermöglicht es, aus Gelenkwinkelpositionen, Gelenkwinkelgeschwindigkeiten und Beschleunigungen, Kräfte und Momente zu berechnen.

- Der rekursive Newton-Euler Algorithmus (RNEA) ist eine synthetische, **impulsbasierte Methode** der Dynamik
- Die physikalischen Größen werden für **jedes einzelne Armelement** betrachtet
- Durch die sich wiederholenden Teilschritte ist dieses Verfahren **effizient und echtzeitfähig**

Newton

Kraft: $f = ma = m\ddot{c}$ (Translation)

Drehmoment: $M = I\dot{\omega}$ (Rotation)

$\ddot{c}$ Beschleunigung Massenschwerpunkt

I Trägheitsmoment

$\dot{\omega}$ Winkelbeschleunigung

$$\begin{pmatrix} n_c \\ f \end{pmatrix} = \begin{pmatrix} I\dot{\omega} + \omega \times I\omega \\ m\ddot{c} \end{pmatrix}$$

Euler

Kreiselwirkung an den Masseschwerpunkten durch Euler- und Zentrifugalkräfte

Drehmoment um Massenschwerpunkt:

$$n_c = I\dot{\omega} + \omega \times I\omega$$

ω Winkelgeschwindigkeit

1

Geschwindigkeiten und Beschleunigungen der einzelnen Armelemente berechnen

$$v_i = v_{p(i)} + \Phi_i \dot{q}_i$$

$$a_i = a_{p(i)} + \Phi_i \ddot{q}_i + \dot{\Phi}_i \dot{q}_i$$

2

Bewegungsgleichung aufstellen, um Kräfte der Armelemente zu erhalten

$$f_i^a = I_i a_i + v_i \times I_i v_i$$

3

Summieren aller räumlich wirkenden Kräfte auf jedes Armelement

$$f_i^a = f_i^e + f_i - \sum_{j \in c(i)} f_j$$

4

Errechnung des jeweiligen Momentenvektors eines Gelenks

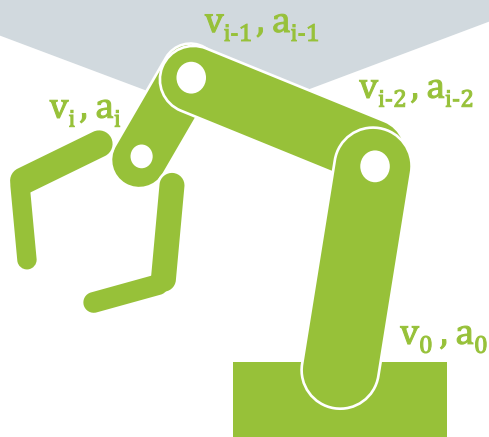
$$\tau_i = \Phi_i^T f_i$$

Quelle: Siciliano

Im Newton-Euler-Verfahren wird der Aufbau des Roboters vorwärts und rückwärts durchlaufen um die wirkenden Beschleunigungen und Kräfte an den einzelnen Elementen zu ermitteln.

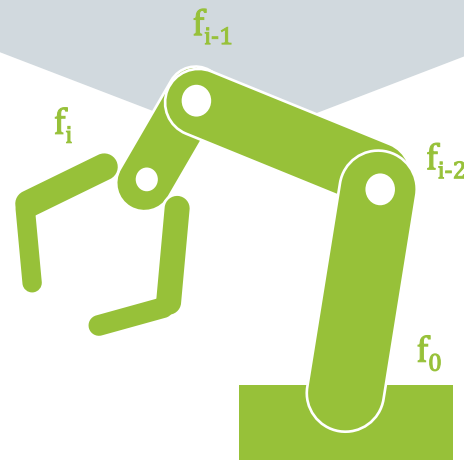
1. Geschwindigkeiten und Beschleunigungen ermitteln

- Beginn an der Roboterbasis
Geschwindigkeit $v_0 = 0$ und Beschleunigung $a_0 = -g$
- Für jedes Armelement von der Basis bis zum Endeffektor (vorwärts) unter Berücksichtigung der Geschwindigkeit/ Beschleunigung des Vorgängers



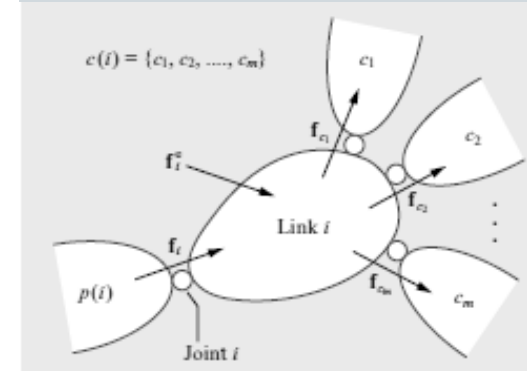
2. Kräfte zum Erreichen der Beschleunigungen berechnen

- Aus den Beschleunigungen kann man Kräfte ableiten, die am Gelenk wirken müssen
- Vorwärtsiteration



3. Kräfte aufsummieren über das Gesamtsystem

- Berechnung der räumlich wirkenden Kräfte
- Zusätzlich greifen externe Kräfte durch z. B. Werkstücke an
- Außerdem werden alle anliegenden Armelemente aufsummiert und mit betrachtet
- Beachten der FHG der jeweiligen Gelenke



4. Momentenvektor berechnen

- Aus den räumlich wirkenden Kräften kann der Momentenvektor abgeleitet werden
- Dieser kann direkt zur Motoransteuerung genutzt werden



Die einfache Implementierung des Newton-Euler-Algorithmus kann über eine koordinatenfrei Darstellung gezeigt werden.

```

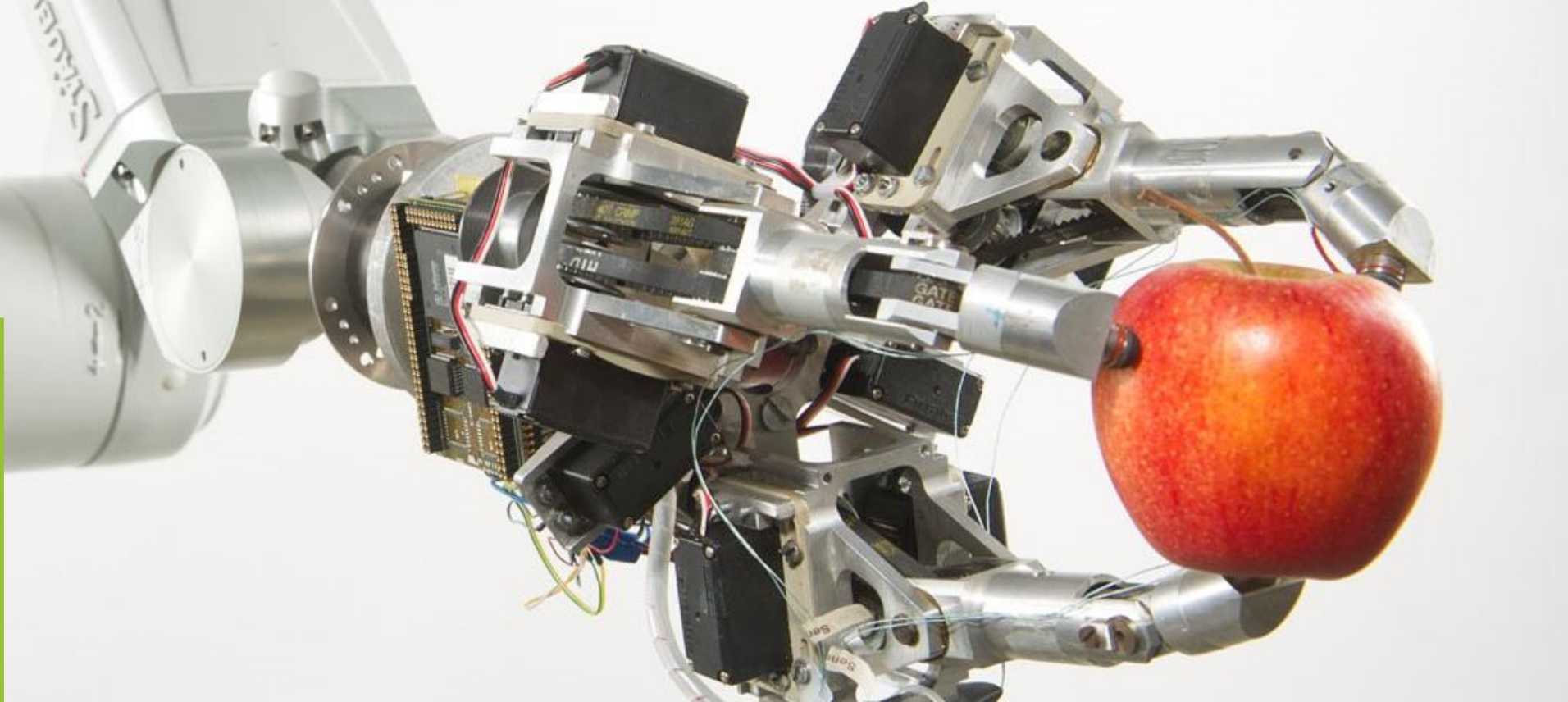
v0=0
a0=ag
for i=1 to NB do
    vi = vp(i) + Φiḡi
    ai = ap(i) + Φiḡi + ḡiḡi
    fia = Iiai + vi × Iivi - fie
end
for i=NB to 1 do
    τi = ΦiTfi
    if p(i)≠0 then
        fp(i) = fp(i) + fi
    end
end
end

```

Dynamikberechnungen über den **Newton-Euler-Algorithmus** oder das **Lagrange-Verfahren** ermöglichen die **Annäherung** an die auftretenden **physikalischen Größen**

- Die **dynamischen Modelle** nähern die vorliegenden Größen an, betrachten jedoch **keine nicht-linearen Effekte** wie Reibung
- Der **Newton-Euler-Algorithmus** eignet sich zur **speicherplatzsparenden** und **echtzeitfähigen** Implementierung
- Anhand der **gegebenen Robotertrajektorie** (Position, Geschwindigkeit und Beschleunigung) werden in der **inversen Dynamik** die Gelenkkräfte und -momente berechnet
- Die **Vorwärtsdynamik** wird dazu verwendet, von den **Gelenkkräften und -momenten** auf die Gelenkbeschleunigungen zurück zu rechnen

Quelle: Siciliano



Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

DANKE